

CEN 571/CSE 598/CSE 494 Lab 3

FPGA Architecture and CAD Exploration using VTR

1 Objectives

1. Learn how to use the VTR framework
2. Explore and compare various FPGA logic block architectures
3. Explore and compare various FPGA routing architectures
4. Modify parameters of FPGA CAD algorithms (placement and routing) and observing their effects on the quality of results

2 Background

VTR is an academic framework for FPGA architecture and CAD exploration. Here's the documentation of VTR: <https://docs.verilogtorouting.org/en/latest/>

We discussed how VTR works in the class. You can browse through the documentation. Some topics of interest could be:

- FPGA architecture description:
 - <https://docs.verilogtorouting.org/en/latest/arch/>
- Command line options to control VPR:
 - https://docs.verilogtorouting.org/en/latest/vpr/command_line_usage/
- Benchmark designs available with VTR:
 - <https://docs.verilogtorouting.org/en/latest/vtr/benchmarks/>

VTR is basically a wrapper over multiple tools:

1. A synthesis tool (this used be ODIN, but now PARMYS is the default synthesis tool)
2. A logic optimization tool and technology mapping tool (called ABC)
3. A packing, placement, and routing tool (called VPR)

Note the difference between VTR and VPR.

VPR is a part of VTR.

VTR is an active open source project. You can browse the codebase here: <https://github.com/verilog-to-routing/vtr-verilog-to-routing>

3 Getting Started

For the labs in this class, you will mainly be using the ASU Research Computing Sol Cluster. Research Computing is a core facility within the Knowledge Enterprise's Research Technology Office dedicated to enabling research, accelerating discovery, and spurring innovation at ASU through the application of advanced computational resources to grand research challenges. Learn

more at researchcomputing.asu.edu. Your account on the Sol supercomputer has been created and is ready to use.

Our supercomputers are accessed using your ASURITE login and password. For uninterrupted access to the supercomputer, please connect to the Cisco AnyConnect Client VPN. If you do not have the Cisco AnyConnect Client VPN already installed, visit sslvpn.asu.edu and follow the installation instructions. For additional details and troubleshooting, please read this document. Once connected to the VPN, you can access the supercomputer through the web portal: The web portal for the Sol supercomputer can be found at <https://sol.asu.edu>. To get a machine/node with an interactive GUI based environment to run your workloads, click on "Interactive Apps" -> "Sol Desktop" and then fill in the form and click "Launch". After a little wait, you will see a link called "Launch Sol Desktop" to open a Linux environment within the browser. Before submitting jobs to the supercomputer cluster, please review the [ASU Research Computing Acceptable Use Policy](#).

We have downloaded and built VTR on the RC nodes. To use this installation, please use the following commands on a terminal of SOL cluster node:

```
> module load vtr/8.0.0-git  
> source activate vtr
```

To confirm that this works, run "which vpr". This should return "/packages/apps/vtr/8.0.0-git/vpr/vpr".

The location of the VTR package is: /packages/apps/vtr/8.0.0-git/

Please note the "-git" in the end.

DO NOT USE /packages/apps/vtr/8.0.0 (i.e. without the -git).

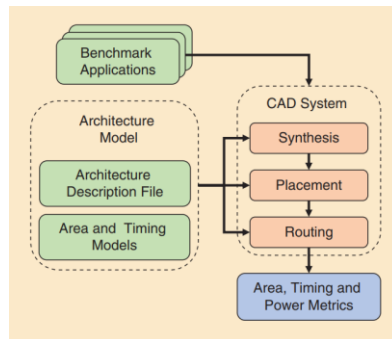
This path is referred to as VTR_ROOT in this document. Please note that anywhere we use the term \$VTR_ROOT in the document, substitute it with the full path of VTR_ROOT. Do not use \$VTR_ROOT as is. It is not an environment variable that we will define.

4 Part 1: Logic Block Architecture Evaluation

4.1 Objective

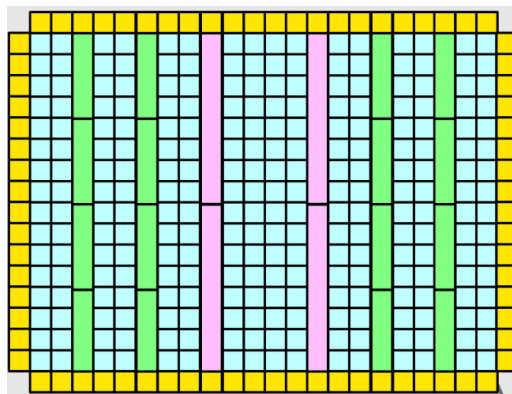
In this part, you will evaluate FPGA architectures with a focus on exploring logic block architectures, using VTR.

The following image gives an overview of the FPGA architecture exploration/evaluation process:



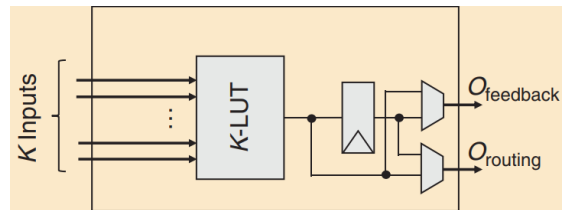
We will use 3 architectures (Arch1, Arch2, Arch3). We will evaluate how much better or worse each is, using the VTR framework.

Here's a logical representation of base architecture you'll be using:



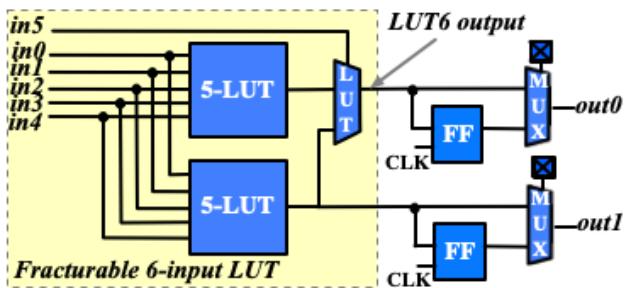
- Yellow = I/O blocks
- Blue = CLBs
- Green = BRAMs
- Pink = DSPs

The Basic Logic Element (BLE) in Arch1 will look like this (K will be 6):

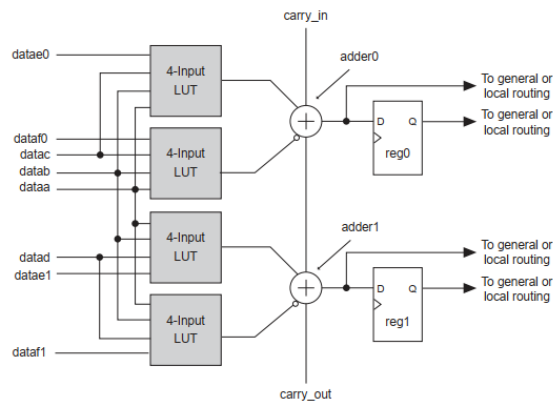


The BLE in Arch2 will have two additional modes, as shown below:

Fracturable 5-LUT mode



Arithmetic mode (with two bit of arithmetic per BLE)



4.2 Exploring FPGA architecture models

Arch1 architecture is expressed in the file **k6_N10_mem32K_40nm.xml**. This file is located in `$VTR_ROOT/vtr_flow/arch/timing/`.

🍀 *Tip: You can simply input path by dragging the directory folder into a terminal window i.e., you can type `cd`, put a space and drag the folder into the terminal window. Then you will move to the directory once you click on the Enter/Return key.*

Let's take a quick walkthrough through the architecture file to see how an FPGA architecture is described.

The topmost tag in every architecture file is the `<architecture>` tag. This tag contains all the information regarding the architecture. Let's take a quick look at the some main aspects of the architecture description:

1) <tiles>

Under this tag, the various types of tiles that exist on this FPGA are specified (these are tile based or island-style FPGA architectures).

```
GNU nano 2.9.8 k6_N10_mem32K_40nm.xml
<port name="data1" clock="clk"/>
<!-- data lines can be broken down into smaller bit widths minimum size 1 -->
<port name="data2" clock="clk"/>
<!-- data lines can be broken down into smaller bit widths minimum size 1 -->
<port name="clk" is_clock="1"/>
<!-- memories are often clocked -->
</input_ports>
<output_ports>
<port name="out1" clock="clk"/>
<!-- output can be broken down into smaller bit widths minimum size 1 -->
<port name="out2" clock="clk"/>
<!-- output can be broken down into smaller bit widths minimum size 1 -->
</output_ports>
</model>
</models>
<tiles>
<tile name="io" area="0">
<sub_tile name="io" capacity="8">
<equivalent_sites>
<site pb_type="io" pin_mapping="direct"/>
</equivalent_sites>
<input name="outpad" num_pins="1"/>
<output name="inpad" num_pins="1"/>
<clock name="clock" num_pins="1"/>
<fc in_type="frac" in_val="0.15" out_type="frac" out_val="0.10"/>
<pinlocations pattern="custom">
<loc side="left">io.outpad io.inpad io.clock</loc>
<loc side="top">io.outpad io.inpad io.clock</loc>
<loc side="right">io.outpad io.inpad io.clock</loc>
<loc side="bottom">io.outpad io.inpad io.clock</loc>
</pinlocations>
</sub_tile>
```

2) <layout>

This tag contains the device grid layout of the architecture. This defines the placement of various tiles.

Commented [K(1)]: This path is for the RC cluster, right?

Commented [K(2R1)]: On the advent lab server:
/mnt/vault0/aaror112/vtr/vtr-verilog-to-routing/vtr_flow/arch/timing/

Commented [AA3R1]: \$VTR_ROOT is defined above in the document. I'll highlight it.

The value of \$VTR_ROOT depends on the machine.

Commented [K(4R1)]: Its path on RC Cluster:
/packages/apps/vtr/8.0.0-git/vtr_flow/arch/timing/

Commented [K(5R1)]: Otherwise,
/packages/apps/vtr/8.0.0/vtr_flow/arch/timing/
Is 8.0.0 and 8.0.0-git directory are identical with each other?

Commented [AA6R1]: No, they are different. Please only use 8.0.0-git.

Commented [K(7)]: The figure is different from the "k6_N10_mem32K_40nm.xml" file on the advent lab server. I will update the figure if needed.

Commented [AA8R7]: Hmm interesting.

I copied this section that introduces the arch file from a workshop I organized last year. We used the same architecture for that workshop, but may be the file has changed now.

Yes, please update these screenshots to match what you see.

Commented [AA9R7]: @KHU KIM (Student), tagging to send a notification. Please update the pictures to match the correct file.

```
GNU nano 2.9.8 k6_N1B_mem32K_40nm.xml

<input name="data" num_pins="64"/>
<input name="we1" num_pins="1"/>
<input name="we2" num_pins="1"/>
<output name="out" num_pins="64"/>
<clock name="clk" num_pins="1"/>
<dc in_type="frac" in_val="0.15" out_type="frac" out_val="0.10"/>
<pinlocations pattern="spread"/>
</sub_title>
</title>
</titles>
<!-- ODIN II specific config ends -->
<!-- Physical descriptions begin -->
<layout>
  <auto_layout aspect_ratio="1.8">
    <!-- Perimeter of 'io' blocks with 'EMPTY' blocks at corners -->
    <perimeter type="io" priority="100"/>
    <corners type="EMPTY" priority="101"/>
    <!-- Fill with 'clb' -->
    <fill type="clb" priority="10"/>
    <!-- Column of 'mult_36' with 'EMPTY' blocks wherever a 'mult_36' does not fit. Vertical offset by 1 for perimeter. -->
    <col type="mult_36" startx="0" starty="1" repeatx="8" priority="20"/>
    <col type="EMPTY" startx="6" repeatx="8" starty="1" priority="19"/>
    <!-- Column of 'memory' with 'EMPTY' blocks wherever a 'memory' does not fit. Vertical offset by 1 for perimeter. -->
    <col type="memory" startx="2" starty="1" repeatx="8" priority="20"/>
    <col type="EMPTY" startx="2" repeatx="8" starty="1" priority="19"/>
  </auto_layout>
</layout>
```

3) <device>

This tag contains the device information such as the type of switch blocks in the architecture and the distribution of the channel width

```
<device>
  <!-- VB & JL: Using Ian Kuon's transistor sizing and drive strength data for routing, at 40 nm. Ian used BPTM
  models. We are modifying the delay values however, to include Metal C and R, which allows more architecture
  experimentation. We are also modifying the relative resistance of PMOS to be 1.8x that of NMOS
  (vs. Ian's 3x) as 1.8x lines up with Jeff G's data from a 45 nm process (and is more typical of
  45 nm in general). I'm upping the Rmin_nmos from Ian's just over 6k to nearly 9k, and dropping
  Rmin_pmos from 18k to 16k to hit this 1.8x ratio, while keeping the delays of buffers approximately
  lined up with Stratix IV.
  We are using Jeff G.'s capacitance data for 45 nm (in tech/ptm_45nm).
  Jeff's tables list C in for transistors with widths in multiples of the minimum feature size (45 nm).
  The minimum contactable transistor is 2.5 * 45 nm, so I need to multiply drive strength sizes in this file
  by 2.5x when looking up in Jeff's tables.
  The delay values are lined up with Stratix IV, which has an architecture similar to this
  proposed FPGA, and which is also 40 nm
  C ipin_cblock: input capacitance of a track buffer, which VPR assumes is a single-stage
  4x minimum drive strength buffer. -->
  <sizing R_min_nmos="8926" R_min_pmos="16067"/>
  <!-- The grid logic tile area below will be used for all blocks that do not explicitly set their own (non-routing)
  area; set to 0 since we explicitly set the area of all blocks currently in this architecture file. -->
  <area grid_logic_tile_area="0"/>
  <chan_width distr>
    <x distr="uniform" peak="1.000000"/>
    <y distr="uniform" peak="1.000000"/>
  </chan_width distr>
  <switch_block type="wilton" fs="3"/>
  <connection_block input_switch_name="ipin_cblock"/>
</device>
```

4) <segmentlist>

This tag contains a group of <segment> tags that specify the types of wire segments and their properties.

```

<segmentlist>
  <!-- VB & JL: using ITRS metal stack data, 90 nm half pitch wires, which are intermediate metal width/space.
        With the 90 nm half pitch, such wires would take 66 um of height, vs. a 90 nm high (approximated as square) Stratix IV tile.
        Reasonable. Using a tile length of 90 nm, corresponding to the length of a Stratix IV tile if it were square. -->
  <segment freq="1.000000" length="4" type="unidir" Rmetal="181" Cmetal="22.5e-15">
    <mux name="0"/>
    <sb type="pattern">1 1 1 1 1</sb>
    <ch type="pattern">1 1 1 1</ch>
  </segment>
</segmentlist>

```

5) <complexblocklist>

This tag contains information about the physical blocks on the chip (that are present in the tiles).

```

<complexblocklist>
  <!-- Define I/O pads begin -->
  <!-- Capacity is a unique property of I/Os, it is the maximum number of I/Os that can be placed at the same (X,Y) location on the FPGA -->
  <!-- Not sure of the area of an I/O (varies widely), and it's not relevant to the design of the FPGA core, so we're setting it to 0. -->
  <pb_type name="io">
    <input name="inpad" num_pins="1"/>
    <output name="inpad" num_pins="1"/>
    <clock name="clock" num_pins="1"/>
  <!-- I/Os can operate as either inputs or outputs.
        Delays below come from Ian Kuon. They are small, so they should be interpreted as
        the delays to and from registers in the I/O (and generally I/Os are registered
        today and that is when you timing analyze them.
        -->
  <mode name="inpad">
    <pb_type name="inpad" blif_model="input" num_pb="1">
      <output name="inpad" num_pins="1"/>
    </pb_type>
    <interconnect>
      <direct name="inpad" input="inpad.inpad" output="io.inpad">
        <delay constant max="4.243e-11" in_port="inpad.inpad" out_port="io.inpad"/>
      </direct>
    </interconnect>
  </mode>
  <mode name="outpad">
    <pb_type name="outpad" blif_model="output" num_pb="1">
      <input name="outpad" num_pins="1"/>
    </pb_type>
  </mode>

```

Arch2 architecture has fracturable LUTs and also adds hard 1-bit adders in a logic block. This architecture is modelled in the file: `k6_frac_N10_frac_chain_depop50_mem32K_40nm.xml`, located in `$VTR_ROOT/vtr_flow/arch/timing`.

The name of each architecture file gives us an idea of the architecture. The number after “k” at the start defines the LUT size in the architecture. In both these files we see that it contains 6 input LUTs. The number after “N” defines the number of LUTs present in a CLB (here, 10). The “mem” string defines the BRAM size present in this architecture (here, 32 Kilobits). In the second architecture, we notice “frac” written between the k6 and N10. This implies that the LUTs present in this architecture are fracturable. And the word “chain” refers to the architecture having carry chains between CLBs (implying there are hard 1-bit adders inside the CLB as well).

Create a folder in your scratch directory. Let’s call it `/scratch/blah/blah/lab3/part1`. Now, copy the Arch1 XML file and Arch2 XML file into this directory.

Now create one more architecture, by copying Arch1 file and modifying it. Let’s call it Arch3. This will be similar to Arch1 but with N=5, I=20 and O=5. I is the number of inputs to the logic block, and O is the number of outputs. Change the area of the “clb” tile (i.e. the logic block) to half of the area in Arch1 (because we have reduced the number of LUTs in the logic block by half). You may have to make other changes for this architecture to work correctly, e.g. changing the local crossbar input and output connections.

Open Arch2 and answer the following for this architecture in your report:

- What technology node (nm) for this architecture?
- Name of the CLB tile
- Name and width of input and output ports of the CLB

Commented [K(10)]: I see "k6_frac_N10_frac_chain_depop50_mem32K_40nm.xml" instead of "k6_frac_N10_frac_chain_depop_mem32K_40nm.xml". Maybe they are identical files?

Commented [K(11R10)]: From this directory: "/packages/apps/vtr/8.0.0-git/vtr_flow/arch/timing/"

Commented [K(12R10)]: Same in "/packages/apps/vtr/8.0.0/vtr_flow/arch/timing/"

Commented [AA13R10): Fixed

- Name and number of BLEs in this CLB
- Names of the modes it supports
- What is the setup time and the clock-to-q delay of the FF in the BLE?
- Does this CLB have a local interconnect? If so, what is the type of local interconnect used in this CLB?
- Name of the memory tile
- What are the various modes supported by this memory
- Name of the DSP tile (is it really called DSP?)
- What are the various modes supported by this DSP?
- What is the grid size (number of tiles vertically and horizontally) of the FPGA? Is this specified in the architecture file?
- What is the routing channel width in the FPGA? Is this specified in the architecture file?
- Wire segments of which length present in the routing channel?
- What of the value of F_s , F_{cin} and F_{cout} in this architecture?
- What does the word “depop” mean? You may have to search in the file to find the answer.

Extra Credit (3%):

Draw a figure showing the structure of the CLB in this architecture file. You can hand draw it. Show the various modes of the BLE. Show the inputs and outputs of the BLE in each mode. Show the inputs, outputs and crossbar of the CLB. Show names of the ports on any block you draw and show widths of the ports. No need to show the internals of the LUT or the crossbar. No need to show all the BLEs in the CLB. Show only 1 and then show that there are N of these.

4.3 Setting and configuring a task

We will be running some benchmarks on these architectures. One way to do this is by running each benchmark through the VTR “flow” on both architectures. The word “flow” here has a specific meaning - it refers to a script called `“run_vtr_flow.py”` which runs VTR on a specified benchmark and a specified architecture file and generates area and timing reports. **We will NOT be using this method in Part 1 of the lab.**

The other way to do this will be to pack architectures and benchmarks into a “task” and run the task which runs VTR on all the specified benchmarks on all the specified architectures in the task configuration file. You can also specify various parameters or options to be used for VTR in the task file. A “task” is run using the `“run_vtr_task.py”` script. **This is the method we will use in Part 1 of the lab.**

For this part of the lab, we will be using VTR benchmarks:

<https://docs.verilogtorouting.org/en/latest/vtr/benchmarks/#vtr-benchmarks>

We will not use all of the benchmarks to reduce runtime.

Find the following paper online:

K. E. Murray et al., “VTR 8: High-performance CAD and Customizable FPGA Architecture Modelling,” ACM Trans. Reconfigurable Technol. Syst., vol. 13, no. 2, pp. 1–55, Jun. 2020

Go to Table 26 (**VPR 8.0: VTR Benchmarks (Relative to VTR 7)**) and identify the list of benchmarks that have number of netlist primitives less than 15,000.

Note: In some versions of the paper, this is Table 27.

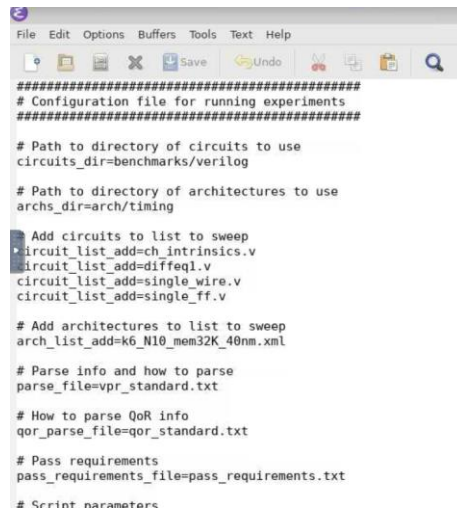
These benchmarks are located in `$VTR_ROOT/vtr_flow/benchmarks/verilog`

Commented [K(14)]: /packages/apps/vtr/8.0.0-git/vtr_flow/benchmarks/verilog/

Create a folder in your home directory. Let’s call it `/home/blah/lab3/part1/part1_task`

Now, create a directory called “config” in this directory and copy the following file into this directory:

`$VTR_ROOT/vtr_flow/tasks/regression_tests/vtr_reg_basic/basic_timing/config/config.txt`



```
#####
# Configuration file for running experiments
#####

# Path to directory of circuits to use
circuits_dir=benchmarks/verilog

# Path to directory of architectures to use
archs_dir=arch/timing

# Add circuits to list to sweep
circuit_list_add=ch_intrinsics.v
circuit_list_add=diffeql.v
circuit_list_add=single_wire.v
circuit_list_add=single_ff.v

# Add architectures to list to sweep
arch_list_add=k6_N10_mem32K_40nm.xml

# Parse info and how to parse
parse_file=vpr_standard.txt

# How to parse QoR info
qor_parse_file=qor_standard.txt

# Pass requirements
pass_requirements_file=pass_requirements.txt

# Sprint parameters
```

Now edit this file:

- Update the correct path of the `circuits_dir`. Change it to be the absolute path of the folder containing the benchmarks you will be running (`$VTR_ROOT/vtr_flow/benchmarks/verilog`).
- Update the correct path of the `archs_dir`. Change it to be the absolute path of the folder containing the architectures you will be comparing between

Commented [K(15)]: /packages/apps/vtr/8.0.0-git/vtr_flow/benchmarks/verilog/

Commented [K(16)]: /packages/apps/vtr/8.0.0-git/vtr_flow/arch/timing/

`($VTR_ROOT/vtr_flow/arch/timing)`. This should be the path of the folder you have the 3 arch files in.

- Delete the circuit files listed under the “**Add circuits to list to sweep**” section and add the file names of the benchmarks you identified from the paper above.
- Delete the architecture(s) listed under the “**Add architectures to list to sweep**”. Add the file names of the architectures we are evaluating for this lab. That is, the 3 architectures discussed earlier.

Lastly in the “**Script parameters**” section, we will make changes so that all the circuits run on the same channel width instead of leaving it up to the VTR to find the minimum channel width required for each benchmark. The rationale behind this is that for this part of the lab, we are not focusing on routing architecture. We just want to keep the channel width fixed to a value (and 300 is a common routing channel width in FPGAs). Remove the existing lines in the “**Scripts parameters**” section and change it to the following:

```
#Script parameters
script_params=-track_memory_usage -route_chan_width 300
```

Keep other parts of the file the same.

You can see this file to read more about tasks:

<https://docs.verilogtorouting.org/en/latest/vtr/tasks/#vtr-tasks>

4.4 Running the task

Now we are ready to run the task we created. When the task is run, it runs VTR on all the benchmarks listed in the config file on all the architectures listed in the config file, and generates reports that contain important metrics like area, frequency, etc. These metrics are useful for comparing architectures on their performance for each benchmark.

To run VTR, go to `the /home/blah/lab3/part1 directory`. Make sure you have loaded the VTR module and enable mamba environment as mentioned in the **Getting Started** section above.

Once the VTR module is loaded and the vtr environment is activated, you need to install the Python package called **prettytable**. To install this package:

```
pip install prettytable
```

Then run:

```
$VTR_ROOT/vtr_flow/scripts/run_vtr_task.py part1_task/
```

Note: You have to replace \$VTR_ROOT with the full path of the VTR installation.

4.5 Observing results

Once this run finishes, observe the contents of the `part1_task` directory. See the files and folders generated during the run.

VTR runs multiple tools in sequence for each benchmark and architecture combination.

In your report, answer the following questions:

- What is the name of the synthesis tool, and the file name of the log file generated by the synthesis tool? What is the command line that was used to run this tool by going through this logfile? What is the name of the output file from this stage of the flow?
- What is the name of the logic optimization and technology mapping tool, and the file name of the log file generated by the synthesis tool? What is the command line that was used to run this tool by going through this logfile? What is the name of the output file from this stage of the flow?
- What is the name of the tool that does packing, placement and routing? What is the file name of the log file generated by the synthesis tool? What is the command line that was used to run this tool by going through this logfile? What is the name of the output file from this stage of the flow?

You will see a file called **"parse_results.txt"** in the `/home/blah/lab3/part1/part1_task/runXXX` folder. This file contains a summary of the results for all architectures and benchmarks that were run by the task. This is a tab separated value file. You can open this file in a spreadsheet software (Excel, Google Sheets or LibreOffice Calc).

In this file we see that the first column gives us the FPGA architecture name. The second column gives us the benchmark that ran on it. Other columns contain the various metrics like run time of the tool, number of clbs used, critical path delay, etc.

You can report the critical path delay using one of two approaches.

First, modify the task configuration file by changing `parse_file = vpr_standard.txt` to `parse_file = vpr_fixed_chan_width.txt`. After making this change, the critical path delay values will be generated and can be included in your report.

Alternatively, note that `vpr_standard.txt` references two timing files:

`timing/vpr.route_min_chan_width.txt` and `timing/vpr.route_relaxed_chan_width.txt`. Since Part 1 uses a fixed channel width, you can instead include `timing/vpr.route_fixed_chan_width.txt` in place of the relaxed channel width file to obtain the critical path delay (`crit_path*`) columns.

Either method is acceptable for reporting the critical path delay in Part 1.

We can compare the architectures by comparing the relevant metrics for these benchmarks for both architectures. To do this, look for the columns in this file that have the relevant metric information.

The metrics we will care about are:

- Area (used logic area + total routing area)
- Critical path delay (which is $1/\text{frequency}$)

Commented [K(17)]: I see the "parse_result.txt" in run001, run002... directory.

Commented [K(18R17)]: /part1/part1_task/run001

Commented [AA19R17]: Fixed.

Parsing information from log files

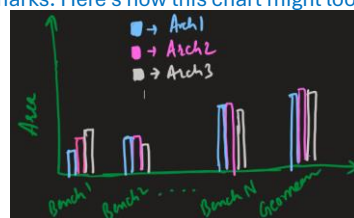
Note that there is a script within VTR that parses log files generated when VTR runs (e.g. vpr.out or yosys.log) and generated this file (parse_results.txt). The content this script parses is controlled by the parse config file that you can specify in the task config file (see the last part of the task config file). There are multiple ways to obtain the data we need from VTR:

- One is you can open the log files and manually look for the data/metrics you're looking for and record it (say in a spreadsheet). This involves a lot of manual effort, especially when you have many 100s of log files.
- The second is you can open the parse_results.txt file that is automatically generated after the tool runs. But it will only have the data/metrics that the default parse config file asks for.
- The third is you can edit the parse config file to specify the kind of data/metrics you are looking for. Then run the tool. Then the parse_results.txt file will have the data/metrics you configured in the parse config file. But there are limitations in the syntax allowed in parse config file, which may not allow you to collect all the specific data/metrics you want to collect.
- The fourth is you write your own parsing scripts that parse the log files. This is the most flexible because you can code it whichever way you want.

For this lab, we are asking you to use the fourth option. For this, you will need to write a Python script which will take the input files (e.g. VPR log files or Yosys log files) and will generate a csv of all the metrics you need to extract. You will need to run VTR for many benchmarks, many architectures, many seeds. So, use this script to perform all sorts of data manipulation that you need (e.g. calculating geometric means across benchmarks, or arithmetic means across seeds, etc.). You are allowed to use LLMs for coding this Python script. Writing this script will make data collection for this lab much easier (last time, manual data collection was the largest complaint about this lab from students). You can have one script for all parts of the lab, or separate scripts for each part. You will submit these scripts along with your report.

In your report:

1. Draw a chart or table comparing the area consumed by each benchmark for all 3 architectures. Along with each benchmark, include a data point for the **geometric** mean of the area across all benchmarks. Here's how this chart might look:



2. Draw a chart or table comparing the frequency obtained by each benchmark for all 3 architectures. Along with each benchmark, include a data point for the **geometric** mean of the frequency across all benchmarks.
3. Explain the reason behind what you observe.

Note: The geometric average is chosen to equally weight all benchmarks; see P. Fleming & J. Wallace, “How Not to Lie with Statistics: The Correct Method to Summarize Benchmark Results,” *Communications of the ACM*, March 1986 for details on why the geometric average is best for such a comparison.

Note: In this part of the lab, you are not asked to run the experiments using multiple seeds. Just one seed (default setting) is enough.

Note: We are using what is called “auto layout”. That means that VPR expands the FPGA layout (i.e. increases the number of height and width of the FPGA grid in it) until a benchmark fits this architecture.

Note: Note the difference between “total logic area” and “used logic area”. The total logic area is the area of the all the logic blocks on the FPGA chip/architecture. The used logic area is the area of the logic blocks to which some operation is actually mapped on to (i.e. the blocks that are actually used by a design or benchmark). We need to use “**used logic area**” for this lab.

Note: A similar thing is valid for routing area. The VPR report has “total routing area”. That tells the total routing area on the FPGA chip/architecture. The used routing area is the routing area which is actually used by the circuit or design or benchmark being mapped on to this FPGA. The “total routing area” is available in the VPR report, but the used routing area is not. The VPR log also specifies the routing area per tile. The used routing area can be estimated by adding the routing area of all tiles that have at least one operation mapped to them. However, for this lab, it is okay to use the “**total routing area**”.

5 Part 2: Routing Architecture Evaluation

5.1 Objective

In this part, you will evaluate FPGA architectures, with a focus on routing architecture, using VTR.

5.2 Setup

Create a folder for this part of the lab. Let's call it `/home/lab3/part2`

In this part of the lab, you will be using a set of benchmarks that are already synthesized and tech mapped. These are provided to you in a zip file. In the `./a4_benchmarks/` directory in the zip file, there are 8 circuits that contain only LUTs, FFs and IO blocks, and have been synthesized to target 6-input LUTs. These circuits are several thousand LUTs in size; they are smaller and simpler than what we would ideally use for architecture experiments, but that lets the CAD tools run quickly. Unzip these benchmark circuits into your `part2` directory.

You will be using the architecture file `k6_N10_40nm.xml`. This file is located in `$VTR_ROOT/vtr_flow/arch/timing`. This architecture describes an architecture where each logic block consists of ten 6-LUTs and 10 FFs (i.e. $N = 10$), with 40 inputs (i.e. $I = 40$) to the logic block, and a full crossbar between the cluster inputs and the look-up table inputs.

Note that these benchmarks are already synthesized and technology mapped. The files we are giving you have the .BLIF extension. This is a human readable netlist file format developed by UC Berkeley. These netlists can be directly given to VPR (the packing, placement and routing tool). We do not need to run synthesis and technology mapping.

In this part of the lab, you will just run VPR (the packing, placement and routing tool in VTR). VPR can automatically find the minimum routing channel width required to route a circuit for a given FPGA architecture. We define a low stress routing as one where the channel width is 30% higher than the minimum required to route the circuit. This is used because it is more realistic to measure routing area and critical path delay with some spare routing in the device.

There are two ways you can run VPR:

- 1) The `vpr` executable is at `$VTR_ROOT/vpr/vpr`. You can directly run this with command line options listed here: https://docs.verilogtorouting.org/en/latest/vpr/command_line_usage/

If you do choose this option, note that VPR produces several output files for each circuit (named after that circuit), and by default these go in the working directory from which you started `vpr`. So, do not run multiple copies of VPR on the same benchmark in the same working directory at the same time, or you can get errors as the various copies of VPR try to write to the same file simultaneously.

Additionally, you will need to parse the metrics you need from the VPR log files. You'll likely write a script (e.g. a python script) for this, otherwise it'll be too difficult to do this by hand.

You will also need to run VPR manually with a "low stress routing channel width". That is, you will run VPR two times – first to find the minimum channel width. Say, this gives the answer W . Then you'll run VPR again with routing channel width of $1.3W$. To ask VPR to run with a specific channel

width, you can use the `--route_chan_width` command line option. For single-driver / unidirectional routing (which we are using in this lab), the routing channel width must be divisible by 2, so round off to the nearest even number.

- 2) You can use the `run_vtr_task.py` script like you used earlier. You will configure the task config a little bit differently:
 - a) Instead of providing the `.v` file in the task file (in `circuit_list_add`), you will provide the `.blif` file. Essentially, we are telling the flow to use this netlist (`.blif` file) as the benchmark.
 - b) You will add `"-starting_stage vpr"` to `script_params` in the task config file. This ensures that synthesis and tech mapping are not run. That's because these benchmarks are already synthesized and tech mapped.
 - c) You will remove `"-route_chan_width 300"` from the task config file. This ensures that the tool find the best minimum channel width first and then automatically re-run the flow with low stress routing channel width.

This may be an easier method and therefore we recommend using this method. That's because this flow already parses the log files generated and generates the `parse_results.txt` file with many metrics. If you don't see the metrics you need, then you will need to provide a different `"parse_file"` in the task config. You will see that the `parse_file` is by default `"vpr_standard.txt"`. This file is located in `$VTR_ROOT/vtr_flow/parse/parse_config/`. Open this file and see how it works. You may need to write a new parse config file.

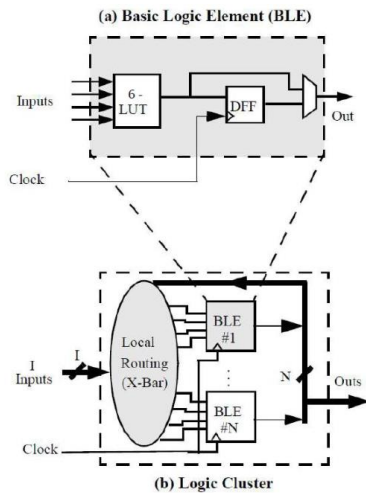
When you use this method, VPR is automatically run twice. First, it'll run to find the minimum channel width. Then it'll be automatically launched with 1.3x the routing channel width to perform low-stress routing. **The vpr log file for the second step is called `vpr.crit_path.out`, and the metrics from the second step in the `parse_results.txt` file have the string `"crit_path"` in their name.**

All the results you report should be geometric averages over the 8 benchmark circuits. The geometric average is chosen to equally weight all 8 benchmarks; see [P. Fleming & J. Wallace, "How Not to Lie with Statistics: The Correct Method to Summarize Benchmark Results," Communications of the ACM, March 1986](#) for details on why the geometric average is best for such a comparison.

To reduce "CAD noise" in your results, you should run the results across multiple random seeds, and then average (arithmetic mean) them. Seeds are basically starting points for CAD tool algorithms. Any integer value can be used as a seed. **Averaging over 5 seeds for each result is best but averaging over 3 seeds is okay too.** To specify a seed, you can use a command line option. If you are running vpr directly, you can specify a seed using `"--seed N"`. If you're using the `run_vtr_task.py`, then you can specify a seed using `"-s --seed N"` in the config file in front of `script_params_list_add`. N is the seed value you want to use. The default is 1 and any integer is a valid and equally good option. You include this averaging into the python scripts.

Note: You will calculate arithmetic average across seeds, but geometric mean across benchmarks. Let's explain this with a simple example consider just one metric here (say critical path delay). You have 5*8 values of the critical path delay, where 5 is the number of seeds and 8 is the number of benchmarks. First, for each benchmark, calculate the mean (arithmetic) of the critical path delay over the seeds. That is, you find the arithmetic of 5 values for each benchmark. Now, you have 8 values of critical path delay

(one for each benchmark). Then, across benchmarks, calculate the mean (geometric) of the critical path delay. Now, you have 1 value of critical path delay.



The 3 metrics you are asked to use in this part of the lab are obtained from the following lines in vpr log file:

```
...
Circuit successfully routed with a channel width factor of 106.
...
Routing area (in minimum width transistor areas)...
Total routing area: 2.61158e+06, per logic tile: 6528.
...
Final critical path delay (least slack): 9.29283 ns, Fmax: 107.61 MHz
```

5.3 Routing wirelength study

Make 3 copies of the k6_N10_40nm.xml architecture file in your project folder. Change them by varying the routing wire length. Each architecture file will have wire length (L) of 1 value (1,4,16). So, you should now have 3 architecture files:

1. With all length 1 wires
2. With all length 4 wires
3. With all length 16 wires

Keep the other parameters of the wires like freq, Rmral, Cmetal, etc. the same. Also, keep the wires fully populated. You will need to read VTR's FPGA architecture documentation to understand how to specify these wires. See: <https://docs.verilogtorouting.org/en/latest/arch/reference/#wire-segments>

Run VPR for each benchmark for each architecture and record the following metrics:

- a) minimum channel width
- b) low-stress routing area
- c) low-stress routing critical path delay

Remember to point to these new architecture files, by either updating the path in the config file (if you are using `run_vtr_task.py`) or by updating the path in the command line (if you are using the `vpr` executable directly).

In your report:

- Draw a chart or table showing the geometric average (over the 8 benchmarks). This will show how the 3 metrics listed above change when you change the wire length.
- Explain your results. Why do the metrics vary when the wirelength is changed in way they do?
- Which routing wire length do you consider the best (gives the best combination of area and delay)?

5.4 Block to routing connectivity study

Take the `k6_N10_40nm.xml` architecture file with all length4 wires. Change it to by varying the `Fcin` and `Fcout` for the logic blocks (CLBs). Create 3 new architectures:

- a) `Fcin = Fcout = 0.15`
- b) `Fcin = Fcout = 0.5`
- c) `Fcin = Fcout = 1.0`

Leave the `Fcin` and `Fcout` of the IO block at the default value of 0.15 for this experiment.

Run VPR for each benchmark for each design and record the following metrics:

- a) minimum channel width
- b) low-stress routing area
- c) low-stress routing critical path delay

Note that this architecture has a full local crossbar between the logic block input pins and the BLE/LUT input pins, as shown in Fig. 1. This means the VPR router is allowed to route a signal to any input of the logic cluster rather than to a specific input pin; the crossbar will get the signal to the right BLE. This crossbar inside the cluster also allows the VPR router to start a signal's routing from any logic block output, rather than the output corresponding to the specific LUT or FF that generates the signal. This is called logical equivalence, and it is specified in the architecture file by the lines shown below.

```
<input name="I" num_pins="40" equivalent="full"/> <output name="O" num_pins="10"
equivalent="instance"/>
```

In your report:

- Draw a chart showing the geometric average (over the 8 benchmarks) of the 3 metrics for the various values of `Fcin` and `Fcout`.
- Explain your results – why do `Wmin`, routing area and critical path delay vary (or not vary) with `Fcin` and `Fcout` in the way they do.

Extra Credit (3%):

We have provided you another architecture file called `k6_N10_sparse_crossbar_40nm.xml`. In this file, there is a sparse crossbar for the local routing inside the logic block (clb), reducing the switches needed to implement the local routing by 67%. When using an architecture with sparse local routing like this, it is beneficial to tell VPR to route signals not just to cluster inputs, but all the way through the local routing to the inputs of LUTs. The inputs of a LUT are logically equivalent (the router can re-arrange which input on a LUT each signal goes to as the CAD flow can change the LUT truth table to compensate), so routing all the way to the LUT inputs gives the router extra options that can help compensate for the less flexible local routing. This does come at the cost of a higher routing CPU time. To do this, additional parameters should be added to VPR command line (`--flat_routing on`) to enable flat (combined inter- and intra-cluster) routing and increase the maximum signal rip-up and re-try iterations in order to fully exploit the connectivity inside the cluster.

Rerun the experiment above using this architecture file. Does the choice between a full crossbar and a partial crossbar impact the channel width and the best choice of F_c ?

Graphics

You can try running VPR with graphics on. This will show you a view of the FPGA, logic blocks, routing wires, switches, etc. To do this, go to the directory which contains `vpr.crit_path.out`. Copy the first line of this file. It's the vpr command line that was used. Now paste this command on the terminal and add "`--disp on`" and press enter. A GUI will popup and you can press buttons on it to proceed and to show more information.

6 Part 3: CAD Algorithm Evaluation

6.1 Objective

In this part of the lab, we will evaluate FPGA CAD algorithms using VTR.

6.2 Setup

In this part of the lab, we will use the architecture file `${VTR_ROOT}/vtr_flow/arch/timing/k6_frac_N10_mem32K_40nm.xml`. The architecture file describes an FPGA with ten 6-input LUTs per logic block, columns of RAM and columns of multipliers. The routing segments span 4 logic-block tiles.

Create a directory for this part of the lab. Let's call it `/home/blah/lab3/part3`

For this part of the lab, we will use four benchmarks. Create a folder inside the lab3 folder called "benchmarks" and then copy these benchmarks here.

```
cp ${VTR_ROOT}/vtr_flow/benchmarks/blif/6/alu4.blif benchmarks
cp ${VTR_ROOT}/vtr_flow/benchmarks/blif/6/ex4p.blif benchmarks
```

For the other two, we don't have netlists, we have RTL/Verilog. So, let's run VTR (**using the `run_vtr_flow.py` command this time**) to generate netlists. We will stop VTR after the ABC stage (which does tech mapping). The syntax of `run_vtr_flow.py` is:

```
run_vtr_flow.py <benchmark_name> <arch_name> <other_options>
```

Note that this is only time in this whole lab that we are suggesting to use `run_vtr_flow.py`. At all other places, we recommend using `run_vtr_task.py`.

```
${VTR_ROOT}/vtr_flow/scripts/run_vtr_flow.py
${VTR_ROOT}/vtr_flow/benchmarks/verilog/diffeq1.v
${VTR_ROOT}/vtr_flow/arch/timing/k6_frac_N10_mem32K_40nm.xml -end abc

cp temp/diffeq1.abc.blif benchmarks/diffeq1.blif

${VTR_ROOT}/vtr_flow/scripts/run_vtr_flow.py
${VTR_ROOT}/vtr_flow/benchmarks/verilog/sha.v
${VTR_ROOT}/vtr_flow/arch/timing/k6_frac_N10_mem32K_40nm.xml -end abc

cp temp/sha.abc.blif benchmarks/sha.blif
```

6.3 Placement Study

VPR allows the user to set various Simulated Annealing parameters: the starting temperature, the ending temperature, the rate at which the temperature decreases, the number of moves per temperature, and the initial random seed. We will play with these parameters.

You can either run VPR directly (using the vpr executable) or use the run_vtr_task.py script. If you use the second method, remember to use “-starting_stage vpr” to not run synthesis and techmapping.

For this study, we will not use timing driven placement. Additionally, we will just stop the CAD flow after placement. You don’t need to do routing. Hence, we need to use these options:

- --pack
- --place (prevent routing)
- --place_algorithm bounding_box (minimize wirelength only and ignore timing)
- --RL_agent_placement off (turn off reinforcement learning based placement)

If you are running VPR directly from the command line (using the vpr executable), then you need to add these options directly to the VPR command line. In this case, you will add these options every time you invoke vpr (you will be invoking the vpr executable multiple times for various seeds and for various benchmarks).

Alternatively, **if you are using the run_vtr_task.py method (which is recommended)**, then you need to add these options to the “script_params” in the task config file. In that case, you won’t be repeating these options multiple times. Just add them once inside each config file you are using.

#1

Run VPR once on the benchmarks. Observe the score (cost function) changes with temperature for each benchmark. You will need to look through the VPR log file (located in the common subdirectory; the file name is vpr.out) to get this information. It’s not available in parse_results.txt file.

In your report:

Plot the score (cost function, “Av BB Cost”) versus the temperature (“T”) for each benchmark. Use a scatter plot, with temperature on the x-axis and cost on the y-axis. It looks best if you set the x-axis to reverse order (descending temperature values), and x-axis to log scale. Draw the four benchmarks as four series on the same plot, or use different plots.

#2

For each circuit, run VPR 5 times (with different seeds) with 5 different starting temperatures ($5 * 5 * 4$ circuits = 100 runs in total). You can choose to use 3 seeds instead. In that case, you will have $5 * 3 * 4 = 60$ runs. Be sure to choose some starting temperatures that are low enough to actually impact the placement results (You may need to use starting temperatures considerably less than 1). Find out from the VPR command reference (located at this location: https://docs.verilogtorouting.org/en/latest/vpr/command_line_usage/), how to change the parameters for the placement algorithm. There will be command line options such as -inner_num, -init_t and exit_t. Read about them.

In your report:

For each circuit, report the mean and standard deviation of the final score for each starting temperature. You may use a chart or a table.

#3

Run VPR 5 times for each circuit with different seeds. You can choose to use 3 seeds instead. Let's call these 'normal results'. Now, in `vpr/src/place/place.cpp`, find the `assess_swap()` function to see the code that determines if a move should be accepted or rejected. You will see the function call to `std::exp(-delta_c/t)` that we discussed in class. We want to change this behavior such that swaps will only be done greedily. For this, we have built a modified VTR here:

`/data/courses/class_cse494598cen571spring2026_aaror112/vtr_modified/vtr-verilog-to-routing`

This path is your modified `$VTR_ROOT`, only for this part of the lab. This means:

- If you are using `run_vtr_task.py`, you have to use:
`/data/courses/class_cse494598cen571spring2026_aaror112/vtr_modified/vtr-verilog-to-routing/vtr_flow/scripts/run_vtr_task.py`
- If you are using `vpr`, you have to use:
`/data/courses/class_cse494598cen571spring2026_aaror112/vtr_modified/vtr-verilog-to-routing/vpr/vpr`

Do not copy the `run_vtr_task.py` or `vpr` executable from this location to your local area. Just use the full path above in the command.

Note that you still need to have loaded the `vtr/8.0.0-git` module and activated the `vtr` environment as you did in the earlier parts of the lab. But instead of using the `run_vtr_task.py` (or the `vpr` executable) from the original unmodified path, you should use `run_vtr_task.py` (or the `vpr` executable) from this new modified path.

Also, note that you don't need to change the path of the architecture file or the benchmark file. You only need to change the path of the `run_vtr_task.py` (or the `vpr` executable).

The outputs of the VTR/VPR execution will be in your local area (`/home/blah/lab3/part3/...`). You should not be writing anything to the `/data/courses/` area.

Now, run VPR from this new VTR location 5 times with different seeds on each circuit with the new acceptance function.

In your report:

Mention the difference in the VPR placement code between the original (this is the VPR code from the VTR installation that you used in earlier parts of the lab) and the modified (this is the VPR code from the VTR installation that was modified for this part (see highlighted text above)).

Report the mean and standard deviation of the resulting scores, for each benchmark. Discuss quality differences observed versus the 'normal results'. You may use a chart or a table.

Commented [K(20): Is this in advent lab server?

Commented [AA21R20]: No. This is on SOL

Commented [K(22): Is this in advent lab server?

Commented [AA23R22]: No. This is on SOL

Commented [K(24): Is this in advent lab server?

Commented [AA25R24]: No. This is on SOL

6.4 Routing Study

Note: You have to use the original unmodified VTR installation for the rest of the lab.

#1

VPR implements timing-driven routing using an improved version of the PathFinder approach described in class. Run VPR multiple times with random seeds (use at least 3 seeds) for each benchmark. Use the command line option `--max_criticality 0.5` (see in the documentation what this does). Set the routing channel width (W) (using `--route_chan_width` on vpr command line or in the task config file) to 40 for alu4, 60 for diffeq1, 50 for ex4p, and 60 for sha. **Do not set `--pack` or `--place` or `--place_algorithm` for this study.**

In your report:

Report the post-routing critical path delay and the router run-time for each benchmark averaged (arithmetic mean) over the seeds. You may use a chart or a table.

#2

Now explore the different parameters of the router and try to improve the critical path delay for the four circuits. Remember to use at least 3 seeds. Use the same routing channel width for each design mentioned in #1. Some parameters of the router you can try are listed below, but you are welcome to try other parameters of the router:

- `initial_pres_fac`
- `pres_fac_mult`
- `acc_fac`
- `bb_factor`
- `astar_fac`
- `max_criticality`
- `criticality_exp`
- `router_max_convergence_count`

You can see VPR documentation of what each of these do:

https://docs.verilogtorouting.org/en/latest/vpr/command_line_usage/#

In your report:

Commented [K(26): Are they files with .v or .blif extension?

Commented [AA27R26]: All benchmarks used in this part of the lab are .blif files.

A few were not .blif originally, but there are steps mentioned above to convert them from .v to .blif.

Commented [K(28R26): Thank you. Just wanted to make sure.

Commented [K(29): I believe those are all positive values, right?

Commented [AA30R29]: Yes

Create a chart or table showing the data. Show the data averaged (arithmetic mean) over seeds for each benchmark. Also, show the data averaged (geometric mean) over all benchmarks. Comment on the effect of parameters on the critical path delay and on router run-time.

#3

VPR implements A* routing. Disable A* routing using the using `--astar_fac 0`. Repeat step #1.

In your report:

Create a chart or table showing the data. Show the data averaged over seeds for each benchmark. Also, show the data averaged (geometric mean) over all benchmarks. Comment on how router run-time and quality is affected by disabling A* routing.

Note: Please ensure that you're calculating the averages the right way. Wherever you are asked to calculate means across seeds and across benchmarks, this information is helpful. You will calculate arithmetic average across seeds, but geometric mean across benchmarks. Let's explain this with a simple example consider just one metric here (say critical path delay). You have 5×8 values of the critical path delay, where 5 is the number of seeds and 8 is the number of benchmarks. First, for each benchmark, calculate the mean (arithmetic) of the critical path delay over the seeds. That is, you find the arithmetic of 5 values for each benchmark. Now, you have 8 values of critical path delay (one for each benchmark). Then, across benchmarks, calculate the mean (geometric) of the critical path delay. Now, you have 1 value of critical path delay.

7 Deliverables

Create a zip file containing 3 folders (part1, part2, part3) and a report. Name zip file lab2_<firstname1>_<lastname1>_<firstname2>_<lastname2>.zip and name report lab2_<firstname1>_<lastname1>_<firstname2>_<lastname2>.pdf. Upload these two files to Canvas. See below for details.

Code

Provide all your code (arch xml files, task config files, command files, etc.). Separate them into folders: part1, part2, part3. In each folder, provide a README file as a text file on how to compile/run/verify your code. The TA should be able to reproduce the code.

Parsing Script(s)

Provide the Python script(s) you used to parse results for all parts of the lab.

Report

This should be a PDF file. One file separated into sections. One section for each part. Provide the information asked in each part.

Explicitly in red color, mention anything you've done to get extra credit.

8 How will you be graded

- The correctness, completeness and performance of your solutions/code (50%)
- The correctness, completeness and clarity of your report (50%)
 - Please make sure you include the reasoning/commentary about your observations of the results. If you just put the results in the report without any discussion/commentary/conclusions, you will not get points.